UNIX-like operating systems have numerous powerful utilities accessible via the command-line and shell-scripting, which are flexible enough to operate in a variety of problematic domains. Text processing is one of these. Among the many small, beautiful and efficient tools is Sed (the stream editor). You will love it, once you master the skill of using and writing Sed scripts. This article starts with its basics and goes on to solving different text-processing tasks.

Sed was written for UNIX by Lee E McMahon of Bell Labs in 1974, inspired by Perl's text-processing capabilities. It can be used for simple text operations as well as complex programs. Typical uses of Sed are simple text replacement, testing for sub-strings, complex text replacement with regular expressions, selective printing of text in a file, finding text described by regular expressions in a large text file, and solving text manipulation problems by using data structures like stack and queue.

Sed is available with all GNU/Linux distributions; there are also versions for Mac OS X and MS Windows. You can provide the input to Sed via a filename given as an argument, or via standard input (stdin). Let's look at a few simple uses of Sed that you can try out in a terminal window:

```
$ echo This is a line of text. | sed 's/text/replaced text/'
This is a line of replaced text.
```

In the above one-liner, we passed data into standard input. In '*s/text/replaced text/*', the initial *s* stands for 'substitute'; the '/' is a delimiter. Substitution has two parts: the text to be found, and the replacement text. In this case, '*text*' is the text to find, and '*replaced text*' is what must be substituted; these are also separated by the '/' delimiter.

```
$ seq 10 | sed '/[049]/d'
1
2
3
5
6
7
8
```

The seq command outputs a sequence of numbers, one to a line. For more information, run *man seq*. In the above code snippet, we have a match pattern—the set of numbers 0, 4 and 9—and '*d*' specified for the action of deletion. Every matching line (containing any number from the set) is thus deleted—removed from the output. You can try different sequences—for example, *seq 99 120 | sed '/[049]/d'*, to understand the operation.

A peek into regular expressions

'Regular expressions' (often shortened to 'regex') is a language used to represent patterns for matching text. Regular expressions are the primary text-matching schema in all text-processing tools, including Sed. If you are already familiar with these, you can skip this section.